

04

Mastery

Large-Scale Systems and Technical Leadership

DURATION

24+ weeks

WEEKLY EFFORT

5-10 hours per week

CAPSTONE

Data Streaming Platform

Purpose and expectations

Operate at system scale: reason about consistency, partitioning, consensus, streaming, cloud infrastructure, security, and low-level performance. The objective is independent technical judgment supported by experiments, operational evidence, and clear design communication.

RECOMMENDED DURATION**24+ weeks****WEEKLY COMMITMENT****5-10 hours per week****ENTRY CRITERIA****Level 3 and hands-on ownership of distributed services****PRIMARY PROJECT****Data Streaming Platform****CORE FOCUS****Distributed systems, event sourcing, Kafka, infrastructure as code, cloud security, advanced C++, and technical leadership**

How to use this roadmap

1. Study for understanding, then validate the idea through implementation.
2. Keep evidence: source code, tests, measurements, diagrams, and short decision records.
3. Prefer a working, explained trade-off over a large but unfinished feature set.
4. Complete each assessment from a clean checkout before advancing.

Capabilities at completion

- Use CAP and PACELC to reason precisely about consistency, availability, partitions, and latency.
- Estimate demand and produce complete system designs with requirements, data models, bottlenecks, and trade-offs.
- Explain Raft, quorum, leader election, log replication, and the purpose of consensus protocols.
- Build event stores, projections, snapshots, CQRS flows, and idempotent saga handlers.
- Design Kafka topics, partitions, consumer groups, replay, delivery semantics, and stream processing.
- Provision VPC, managed Kubernetes, streaming, and database infrastructure with Terraform and protected remote state.
- Apply OAuth 2.0/OIDC, mTLS, secrets management, least privilege, and OWASP controls.
- Use memory-mapped I/O, profiling, and careful benchmarks to improve C++ data paths.
- Understand the complexity and safety requirements of lock-free data structures.
- Lead design reviews, communicate risks, and turn production evidence into technical priorities.

Roadmap at a glance

PHASE	SCHEDULE	FOCUS	EVIDENCE
Distributed foundations	Weeks 1-3	CAP, PACELC, design, consensus	System design portfolio
Event models	Weeks 4-6	Event sourcing, CQRS, Kafka	Event-driven core
Streaming capstone	Weeks 7-12	C++, Kafka, TimescaleDB, Terraform, K8s	Data Streaming Platform
Performance and security	Weeks 13-16	mmap, OWASP, OAuth, mTLS, lock-free structures	Hardening and assessment
Continuous mastery	Week 17 onward	Papers, open source, leadership	Self-directed development

Completion rule

Do not treat elapsed time as completion. Advance when you can demonstrate the deliverables, explain the design decisions, and reproduce the result.

Weekly learning plan

WEEK 01**System Design with CAP and PACELC**

5h study + 3h practice

LEARNING FOCUS

- Consistency, availability, and partition tolerance without the misleading 'pick any two' shortcut.
- PACELC trade-offs during partitions and normal operation.
- Strong, eventual, causal, and read-your-writes consistency.
- Quorum relationships such as $R + W > N$, including their assumptions.

APPLIED WORK

- Design a banking system, news feed, and chat system.
- Identify the partition behavior and normal-operation trade-off for each.
- State what is guaranteed, what can be stale, and how failures surface.

DELIVERABLE Three concise design briefs with workload assumptions, CAP/PACELC analysis, failure behavior, and explicit trade-offs.

WEEK 02**System Design Practice: Social, Video, and Mobility**

3h study + 5h practice

LEARNING FOCUS

- Functional and non-functional requirements.
- DAU, throughput, bandwidth, storage, and growth estimates.
- Data models, high-level architecture, hot paths, and bottlenecks.
- Trade-off communication under uncertainty.

APPLIED WORK

- Design a Twitter or Threads-style feed.
- Design a YouTube-style video platform.
- Design an Uber-style dispatch flow or a mini Dynamo-style key-value store.
- Review each design for failure modes and observability.

DELIVERABLE A system-design-practice repository with three complete, diagrammed designs and written trade-offs.

WEEK 03**Raft and Distributed Consensus**

4h study + 3h practice

LEARNING FOCUS

- Leader election, terms, log replication, commit rules, and safety.
- Membership changes and failure recovery at a conceptual level.
- How Raft and Paxos differ in presentation and use.

APPLIED WORK

- Use a Raft simulator to force elections and partitions.
- Implement a conceptual leader-election model.
- Write down safety invariants and failure assumptions.

DELIVERABLE A visual Raft walkthrough and a small simulation demonstrating election and replicated-log behavior.

WEEK 04 Event Sourcing and Event Stores

4h study + 5h practice

LEARNING FOCUS

- Append-only events, aggregate versions, optimistic concurrency, snapshots, and projections.
- Rehydration, schema evolution, replay, and auditability.
- When event sourcing is an unnecessary cost.

APPLIED WORK

- Implement a PostgreSQL event table and append transaction.
- Reject conflicting aggregate versions.
- Rebuild aggregate state from ordered events.
- Test concurrent writers and replay.

OPTIMISTIC APPEND

```
await client.query('BEGIN');
const { rows } = await client.query(
  'SELECT MAX(version) AS version FROM events WHERE aggregate_id = $1',
  [event.aggregateId],
);
if ((rows[0].version ?? 0) + 1 !== event.version) {
  throw new ConcurrencyError('Aggregate version conflict');
}
await client.query('INSERT INTO events (...) VALUES (...)');
await client.query('COMMIT');
```

DELIVERABLE A tested PostgreSQL event store with optimistic concurrency, aggregate rehydration, and replay documentation.

WEEK 05 CQRS and Choreographed Sagas

3h study + 5h practice

LEARNING FOCUS

- Separate write models, read models, and their operational costs.
- Projection handlers, materialized views, lag, retries, and idempotency.
- Saga compensation and observability.

APPLIED WORK

- Create an Order command handler backed by the event store.
- Build a query-side projection.
- Make projection updates idempotent.
- Measure and expose projection lag.

DELIVERABLE A CQRS slice with command handling, a query projection, replay support, and observable eventual consistency.

LEARNING FOCUS

- Topics, partitions, offsets, leaders, replicas, ISR, and consumer groups.
- Key-based ordering and rebalancing behavior.
- At-most-once, at-least-once, and exactly-once processing boundaries.
- Windowed aggregation with Kafka Streams or ksqlDB.

APPLIED WORK

- Produce events with stable keys.
- Run multiple consumers in one group and observe partition assignment.
- Implement idempotent processing or transactions where required.
- Build a windowed count or average.

DELIVERABLE A replayable Kafka pipeline with documented partitioning, consumer behavior, failure recovery, and processing semantics.

WEEKS 07-12

Capstone: Data Streaming Platform

Build a cloud-oriented event ingestion, processing, storage, query, and observability platform.

LEARNING FOCUS

- High-throughput ingestion and Kafka partitioning.
- C++ processing with bounded memory and windowed aggregation.
- Time-series storage and time-bucket queries.
- Terraform-managed network, Kubernetes, streaming, and database infrastructure.
- Load testing beyond one million events and operational dashboards.

APPLIED WORK

- Build an authenticated Node.js/TypeScript Ingest API.
- Consume and aggregate events in a C++ core using librdkafka.
- Store results in TimescaleDB and expose a Query API.
- Deploy processors and APIs to Kubernetes.
- Track throughput, latency, consumer lag, and errors in Grafana.

ARCHITECTURE AND SCOPE

- Ingest API publishes validated sensor events
- Kafka partitions events by stable key
- C++ Core performs windowed processing
- TimescaleDB stores time-series results
- Query API serves raw and aggregated data
- Terraform provisions VPC, Kubernetes, Kafka, and database resources

DELIVERABLE A working platform with ingestion, Kafka, C++ processing, TimescaleDB, query APIs, Terraform, Kubernetes, dashboards, and documented load-test evidence.

LEARNING FOCUS

- Virtual memory, page cache, page faults, cache locality, mmap, and explicit I/O trade-offs.
- Why memory mapping is not automatically zero-copy end to end.
- Resource lifetime, error handling, and large-file safety.

APPLIED WORK

- Process a large log through read and mmap implementations.
- Benchmark with warm and cold cache conditions.
- Profile CPU, page faults, and throughput.
- Document where each approach wins.

MAPPED FILE LIFECYCLE

```
fd_ = open(path.c_str(), O_RDONLY);
size_ = lseek(fd_, 0, SEEK_END);
mapped_ = mmap(nullptr, size_, PROT_READ, MAP_PRIVATE, fd_, 0);
if (mapped_ == MAP_FAILED) throw std::system_error(errno, std::generic_category());
// Process bytes, then munmap(mapped_, size_) and close(fd_).
```

DELIVERABLE A reproducible read vs mmap benchmark with methodology, profiles, results, and limitations.

LEARNING FOCUS

- OWASP risk categories and defense in depth.
- OAuth 2.0 roles, authorization code with PKCE, and OIDC identity claims.
- Mutual TLS for service identity.
- Secrets management, rotation, least privilege, and auditability.

APPLIED WORK

- Harden headers and CORS with explicit origins.
- Apply strict authentication rate limits and complete input validation.
- Move secrets out of source and into a managed store.
- Document an mTLS trust and rotation model.

DELIVERABLE A security review with threat model, prioritized findings, implemented mitigations, and residual risk.

LEARNING FOCUS

- C++ memory ordering, atomic compare-and-exchange, ABA, reclamation, and progress guarantees.
- Why a correct lock-free queue is more than an atomic linked list.
- When a mutex-based bounded queue is safer and fast enough.

APPLIED WORK

- Study the Michael-Scott queue algorithm.
- Implement or adapt a queue with a correct reclamation strategy.
- Run correctness stress tests before benchmarks.
- Compare against a well-designed mutex-based queue.

DELIVERABLE A correctness-first comparison of lock-free and lock-based queues, including memory-reclamation strategy and benchmark caveats.

WEEK 16 Level Assessment

LEARNING FOCUS

- Explain CAP with realistic failure modes and why CA is not meaningful under network partitions.
- Defend event sourcing boundaries, Kafka partitioning, Terraform state, memory-mapped I/O, and security controls.
- Design a real-time system that processes ten million events per day.

APPLIED WORK

- Answer at least five of seven questions accurately.
- Demonstrate the cloud-deployed Streaming Platform.
- Lead a design review covering scale, reliability, cost, security, and operations.

DELIVERABLE Pass when the assessment threshold is met and the Streaming Platform operates in a cloud environment with reproducible infrastructure and evidence.

WEEK 17+ Continuous, Self-Directed Mastery

LEARNING FOCUS

- Foundational papers: MapReduce, Bigtable, Dynamo, Spanner, and Raft.
- Engineering publications from organizations operating at scale.
- Advanced algorithm and system design practice.
- Open-source contribution and technical leadership.

APPLIED WORK

- Maintain a paper-reading log with critiques, not summaries alone.
- Contribute fixes or features to relevant open-source projects.
- Run periodic design reviews and incident simulations.
- Mentor another engineer and document the learning loop.

DELIVERABLE A continuing evidence portfolio: design reviews, paper critiques, open-source contributions, operational retrospectives, and mentoring outcomes.

Recommended source material

Distributed systems

Designing Data-Intensive Applications; Raft paper and visual guide; Dynamo, Bigtable, Spanner, and MapReduce papers.

Streaming and event models

Kafka architecture and design documentation; Martin Fowler on Event Sourcing and CQRS.

Infrastructure and security

Terraform state guidance; Kubernetes documentation; OWASP; OAuth 2.0 and OpenID Connect specifications.

C++ performance

Linux mmap documentation; C++ memory model references; CppCon performance material; proven concurrency libraries.

Completion perspective

Mastery is not a final checklist. It is the ability to make, test, communicate, and revisit high-impact technical decisions as systems and constraints evolve.